East West University

Department of Computer Science and Engineering (CSE)
Semester: (Spring, Year: 2026), B.Sc. in CSE

Design Pattern Lab

Group: 2

Course Code: *CSE423*
Course Title: *Software Architecture*
Section: 01

Submitted to:

Ahmed Adnan
Lecturer, Department of Computer Science and Engineering

Students Details

Name	ID
Azahar Mahmud Chowdhury Rafi	2021-3-60-019
Sheikh Md Rakibul Hasan	2022-3-60-009
Ahnaf Anan	2023-1-60-089

Submission Date: 7th March, 2026

CREATIONAL PATTERNS

1. Singleton Pattern (Thread-Safe)

The original AppConfig implementation was not thread-safe. If multiple threads accessed getInstance() at the exact same moment initially, they could create multiple instances of the config object, violating the Singleton rule.

We applied the "double-checked locking" pattern with the volatile keyword to ensure that even in a multi-threaded environment, only one instance is ever created.

Before

```java
public class AppConfig {
    private static AppConfig instance; // Not thread-safe

    public static AppConfig getInstance() {
        if (instance == null) {
            instance = new AppConfig();
        }
        return instance;
    }
}
```

After

```java
public class AppConfig {
    private static volatile AppConfig instance; // volatile ensures threads see the updated value

    public static AppConfig getInstance() {
        if (instance == null) {
            synchronized (AppConfig.class) { // Lock only upon first creation
                if (instance == null) {
                    instance = new AppConfig();
                }
            }
        }
        return instance;
    }
}
```

2. Factory Method

Originally, object creation for Student and Instructor was hardcoded using the new keyword scattered across multiple files (like StudentCsvService, StudentManagementHelper, etc.). If the constructor signature changed, we would have to hunt down and fix every file.

We created an abstract PersonFactory with concrete subclasses (StudentFactory, InstructorFactory). Now, creation logic is centralized in one place.

Before (in StudentCsvService):

```
// Hardcoded creation scattered in your code
Student student = new Student(
    Integer.parseInt(parts[0]), parts[1], parts[2],
    parts[3], LocalDate.parse(parts[4])
);
```

After

```
// Centralized construction via Factory
public abstract class PersonFactory {
    public abstract Person createPerson(int id, String fullName, String email, LocalDate dob, String... extras);
}

public class StudentFactory extends PersonFactory {
    @Override
    public Student createPerson(int id, String fullName, String email, LocalDate dateOfBirth, String... extras) {
        return new Student(id, extras[0], fullName, email, dateOfBirth); // extras[0] is regNo
    }
}

// In StudentCsvService:
private final StudentFactory studentFactory = new StudentFactory();
Student student = studentFactory.createPerson(id, name, email, dob, regNo);
```

STRUCTURAL PATTERNS

3. Adapter Pattern

The code to format domains (like Student and Enrollment) into CSV strings was hardcoded directly inside the IO export services (String.format("%d,%s...", ...)). If the school later wanted to export to JSON or XML, you'd have to rewrite the services.

We extracted an ExportFormatAdapter interface and created a CsvExportAdapter. The export service now asks the adapter to format the data, unaware of the actual format.

Before (in StudentCsvService):

```
List<String> lines = dataStore.getAllStudents().stream()
    .map(student -> String.format("%d,%s,%s,%s,%s,%s",
        student.getId(), student.getRegNo(), student.getFullName(),
        student.getEmail(), student.getDateOfBirth(), student.getStatus()))
    .collect(Collectors.toList());
```

After

```
// 1. Interface
public interface ExportFormatAdapter {
    String formatStudentRecord(Student student);
}

// 2. CSV Implementation
public class CsvExportAdapter implements ExportFormatAdapter {
    @Override
    public String formatStudentRecord(Student student) {
        return String.format("%d,%s,%s,%s,%s,%s", student.getId(), student.getRegNo(), ...);
    }
}

// 3. Usage in StudentCsvService
private final ExportFormatAdapter formatAdapter = new CsvExportAdapter();
List<String> lines = dataStore.getAllStudents().stream()
    .map(student -> formatAdapter.formatStudentRecord(student))
    .collect(Collectors.toList());
```

4. Decorator Pattern

We wanted to add auditing/logging to EnrollmentService without modifying its core business logic. Directly modifying it violates the Open-Closed Principle (OCP).

We extracted an interface (EnrollmentServiceInterface), and created a LoggingEnrollmentDecorator that wraps the concrete service, adding pre- and post- logging logic organically.

Before:

```
// In CliManager
private static final EnrollmentService enrollmentService =
    new EnrollmentService(studentService, courseService);
```

After

```
// 1. The Decorator adding logging
public class LoggingEnrollmentDecorator extends EnrollmentServiceDecorator {
    public LoggingEnrollmentDecorator(EnrollmentServiceInterface wrapped) { super(wrapped); }

    @Override
    public void enrollStudent(String regNo, String courseCode) throws Exception {
        System.out.println("[LOG] Attempting enrollment...");
        wrapped.enrollStudent(regNo, courseCode); // Call real service
        System.out.println("[LOG] Enrollment completed.");
    }
}

// 2. In CliManager (Wrapping the service)
private static final EnrollmentServiceInterface enrollmentService =
    new LoggingEnrollmentDecorator(new EnrollmentService(mediator, gradingStrategy));
```

5. Bridge Pattern

In ReportService, the *logic of generating the data* and the *logic of printing it* (System.out) were squashed together. Displaying reports on a GUI or PDF would break it.

Separated into an abstraction (Report) detailing *what* data to get, and an implementor (ReportRenderer) detailing *how* to show it.

Before:

```java
public void generateGpaDistributionReport() {
    Map<String, Long> ranges = getGpaData();
    // Tied permanently to console output
    System.out.println("\n--- GPA Distribution Report ---");
    ranges.forEach((range, count) -> System.out.printf("%s: %d students\n", range, count));
}
```

After

```java
// 1. Renderer Interface (Implementor)
public interface ReportRenderer {
    void renderHeader(String title);
    void renderDataRow(String label, long value);
}
// Concrete: ConsoleReportRenderer implements ReportRenderer

// 2. Report Abstraction
public class GpaDistributionReport extends Report {
    public void generate() {
        Map<String, Long> ranges = getGpaData();
        renderer.renderHeader("GPA Distribution Report");
        ranges.forEach((range, count) -> renderer.renderDataRow(range, count));
    }
}
```

6. Proxy Pattern

The DataStore accepted any data thrown at it (nulls, empty strings) with zero validation. Putting validation in DataStore would bloat the storage layer logic.

A DataStoreProxy wraps the real store, acting as a gatekeeper that validates rules before passing the data to memory.

Before:

```
public class DataStore {
    public void addStudent(Student student) {
        students.put(student.getRegNo(), student); // Crashes if student is null
    }
}
```

After

```
public class DataStoreProxy implements DataStoreInterface {
    private final DataStoreInterface realDataStore;

    @Override
    public void addStudent(Student student) {
        if (student == null) throw new IllegalArgumentException("Null student");
        if (student.getRegNo() == null) throw new IllegalArgumentException("Empty ID");

        System.out.println("[PROXY] Validating...");
        realDataStore.addStudent(student); // Pass to real store only if valid
    }
}
```

BEHAVIORAL PATTERNS

7. Strategy Pattern

The formula determining what marks equal what grade (e.g., 90=S, 80=A) was rigidly hardcoded in the Grade enum. What if the school curves the grade or implements a lenient system?

Extracted into a GradingStrategy interface,
allowing StandardGradingStrategy and RelativeGradingStrategy to be swapped dynamically at runtime without replacing core code.

Before:

```
// Hardcoded enum logic
public enum Grade {
    public static Grade fromMarks(int marks) {
        if (marks >= 90) return S;
        if (marks >= 80) return A; //...
    }
}

// Inside EnrollmentService
enrollment.setGrade(Grade.fromMarks(marks));
```

After

```
public interface GradingStrategy {
    Grade determineGrade(int marks);
}

public class StandardGradingStrategy implements GradingStrategy {
    public Grade determineGrade(int marks) {
        if (marks >= 90) return Grade.S; // ...
    }
}

// Inside EnrollmentService (Strategy Injected)
private final GradingStrategy gradingStrategy;
enrollment.setGrade(gradingStrategy.determineGrade(marks));
```

8. Observer Pattern

When a student successfully enrolls, we currently just call System.out.println. If we want to suddenly send a confirmation email or update school statistics, we shouldn't have to alter EnrollmentService.

The service holds a list of EnrollmentObservers. Once an action happens, it notifyObservers() letting them handle their individual side effects asynchronously.

Before:

```
student.addEnrollment(newEnrollment);
System.out.println("Enrollment successful!"); // Rigid side effect
```

After

```
student.addEnrollment(newEnrollment);
notifyEnrollment(student, course); // Triggers observers recursively

// The notifications logic is offloaded to separate classes:
public class EnrollmentNotifier implements EnrollmentObserver {
    public void onEnrollment(Student student, Course course) {
        System.out.printf("[EMAIL] Dear %s, you enrolled in %s\n", student.getName(), course.getName());
    }
}
```

9. Mediator Pattern

EnrollmentService relied heavily on directly referencing both StudentService and CourseService. As the software scales and relies on Billing or Staffing services, it would become a tangled web of dependencies

Extracted a ServiceMediator. The enrollment service talks *only* to the central mediator to locate what it needs, keeping components fully uncoupled.

Before:

```
public class EnrollmentService {
    private final StudentService studentService; // Tightly coupled
    private final CourseService courseService;   // Tightly coupled

    public EnrollmentService(StudentService s, CourseService c) {
        this.studentService = s;
        this.courseService = c;
    }
}
```

After

```
public interface ServiceMediator {
    Student findStudent(String regNo);
    Course findCourse(String courseCode);
}

// Now EnrollmentService knows NOTHING about CourseService or StudentService!
public class EnrollmentService {
    private final ServiceMediator mediator;

    public void enrollStudent(String regNo, String courseCode) {
        Student student = mediator.findStudent(regNo);
    }
}
```